

Rôle

Rapier, dans un projet **Three.js**, sert principalement à **gérer la physique** : collisions, gravité, mouvements réalistes, forces, frottements, etc.

<https://rapier.rs/javascript3d/index.html>

Le moteur

```
// dans engine/physic.js
import Rapier from '@dimforge/rapier3d-compat'

await Rapier.init()

export default new Rapier.World({ x:0, y:-9.81, z:0})
```

il est intégrée dans app

```
const world = new World(meshes.visuals, meshes.colliders, physic)
```

et dans la class world

```
initPhysic(meshes, physic) {

  for(const mesh of meshes) {
    // 1 Description : corps fixe (ex: un mur, un sol, un décor)
    const rigidBodyDesc = RigidBodyDesc.fixed()
    // 2 Création du rigid body dans le monde
    const rigidBody = physic.createRigidBody(RigidBodyDesc)

    // 3 Description du collider (forme)
    const geo = mesh.geometry

    const vertices = new Float32Array(geo.attributes.position.array)

    const indices = new Float32Array(geo.index.array)

    const colliderDesc = ColliderDesc.trimesh(vertices, indices)
    // 4 Création du collider attaché au rigid body
```

```

        physic.createCollider(colliderDesc, rigidBody)

    }

}

```

Les objets (mesh) sont défini par deux paramètres :

- RigidBody = squelette physique

✓ Static (immobile, décor)

```
RigidBodyDesc.fixed()
```

✓ Dynamic (bouge, subit la gravité)

```
RigidBodyDesc.dynamic()
```

- **Collider = sa peau / son volume**

Comme une statue origami on prend un tableau de points : vertices et indices place les formes entre les points (par facettes triangulaires)

Références :

<https://trimesh.org/trimesh.geometry.html>

Evolution

On exporte une fonction pour créer des objets fixes

```

// tool/function.js
import { ColliderDesc, RigidBodyDesc } from "@dimforge/rapier3d-compat";

function createColliderGeo(geo, rigidBody, physic) {

    const vertices = new Float32Array(geo.attributes.position.array)

    const indices = new Float32Array(geo.index.array)

    const colliderDesc = ColliderDesc.trimesh(vertices, indices)

    return physic.createCollider(colliderDesc, rigidBody)
}

```

```

}

export function createRigidBodyFixed(mesh, physic) {

  const rigidBodyDesc = RigidBodyDesc.fixed()

  const rigidBody = physic.createRigidBody(rigidBodyDesc)

  const collider = createColliderGeo(mesh.geometry, rigidBody, physic)

}

```

- **RigidBodyDesc**
→ Décrit un “corps physique” : dynamique, statique, .
- **RigidBody**
→ Instance réelle dans le monde Rapier.
- **ColliderDesc**
→ Décrit la forme du volume (box, sphere, trimesh, etc.)
- **Collider**
→ Instance réelle du volume attachée au rigid body.
- **createColliderGeo**
→ Fonction utilitaire qui convertit une géométrie Three.js en collider Rapier.
- **createRigidBodyFixed**
→ Fonction exportée créant un rigid body statique et son collider associé.

on résume alors dans world et on crée les objets fixes

```

initPhysic(meshes, physic) {

  for(const mesh of meshes) {

    createRigidBodyFixed(mesh, physic)

  }

}

```

Un player

La grande nuance c'est un objet dynamique. Donc recalculer sa forme exacte est lourde. Une méthode est de l'envelopper d'une sphère. Et laisser les calculs pour sa position

```

function createColliderBall(radius, rigidBody, physic) {

    const colliderDesc = ColliderDesc.ball(radius)

    return physic.createCollider(colliderDesc, rigidBody)

}

export function createRigidBodyEntity(position, physic) {

    const rigidBodyDesc = RigidBodyDesc.dynamic()

    rigidBodyDesc.setTranslation( ... position)

    const rigidBody = physic.createRigidBody(rigidBodyDesc)

    const collider = createColliderBall(0.25, rigidBody, physic)

    return {rigidBody, colliders}

}

```

Un détail comme la position évolue et sera vu plus tard. On est obligé de recréer le player et donc on return pour refaire a chaque frame.

Ainsi dans l'entity player

```

import { Object3D } from "three";

import { createRigidBodyEntity } from "../tool/function"

export default class Player extends Object3D {

    collider = null

    rigidBody = null

    constructor(mesh, physic) {

        super()

        this.position.copy(mesh.position)
    }
}

```

```
    this.initPhysic(physic)

    this.initVisual(mesh)
}

initPhysic(physic) {

    const {rigidBody, collider} = createRigidBodyEntity(this.position, physic)

    this.rigidBody = rigidBody

    this.collider = collider
}

initVisual(mesh) {

    mesh.position.set(0,0,0)

    mesh.castShadow = true

    this.add(mesh)
}

update(){

    this.updatePhysic()

    this.updateVisual()
}

updatePhysic() {

}
```

```
updateVisual() {  
  
    this.position.copy(this.rigidBody.translation())  
  
}  
  
}
```

Pour résumer :

- `Player` = **Object3D** enrichi avec physique et logique
- `physic` = RigidBody + Collider géré par Rapier
- `visual` = mesh affiché dans Three.js et synchronisé avec le rigidBody
- On peut ajouter le player à la scène comme n'importe quel Object3D
- `update()` permet de garder visuel et physique cohérents à chaque frame